

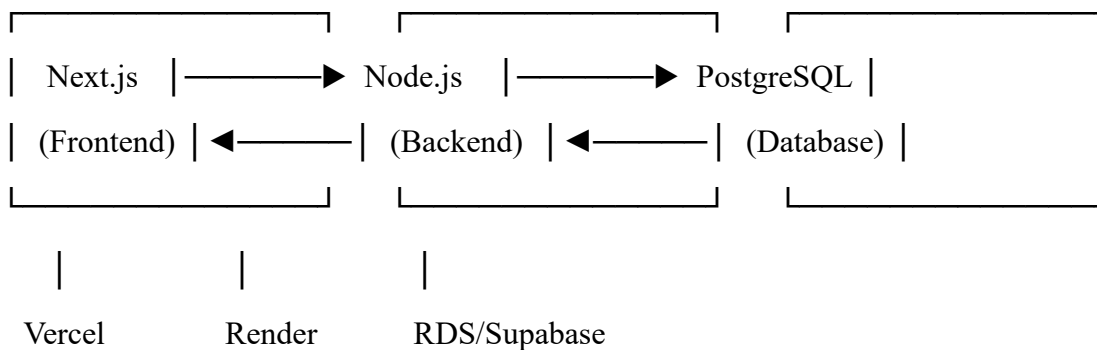
Project Simplified: What This Actually Is

Your friend needs to build a production-ready, multi-tenant financial tracking SaaS (Software as a Service). Think of it like a business version of Mint or Expensify where:

- Multiple companies (tenants) can use the same app
- Each company's data is completely isolated from others
- Users have different roles (Admin, Accountant, Regular User)
- Users can track expenses and income, view dashboards, export data
- The system must handle 10,000+ users simultaneously without crashing

Simplified Architecture Overview

...



...

Complete Project Workflow & Steps

Phase 1: Foundation & Planning (Days 1-3)

Step	Task	Details
1.1	Understand Multi-Tenancy	Learn how to isolate data per organization. Every database query MUST include `orgId`. No exceptions.
1.2	Design Database Schema	Plan tables: `Organization`, `User`, `Transaction`, `Role`, `RefreshToken`. Use foreign keys. No data duplication.
1.3	Set Up Project Structure	Create two repos: `frontend` (Next.js) and `backend` (Express/Node.js).
1.4	Environment Setup	Install Node.js, PostgreSQL, Prisma CLI, set up GitHub repos.

Phase 2: Database & Backend Core (Days 4-10)

Step	Task	Key Requirements
2.1	Initialize Prisma	Set up Prisma schema with all models, relations, and indexes.
2.2	Create Migrations	Run `prisma migrate dev` to generate SQL. Never modify DB manually.
2.3	Implement Authentication	JWT access tokens (15 min expiry) + refresh tokens (rotation strategy). Store tokens securely (HTTP-only cookies, NOT localStorage).
2.4	Role-Based Middleware	Create middleware that checks user roles before allowing API access.
5	Multi-Tenant Middleware	Automatically inject `orgId` into every query based on the logged-in user's organization.
2.6	Transaction CRUD API	Create endpoints for expense/income records with strict validation.
2.7	Financial Consistency	Use Prisma `\$transaction` for any operation that touches money (prevents partial writes).
2.8	Dashboard Aggregation	Build optimized `SUM` and `GROUP BY` queries with proper indexes.
2.9	Filtering & Pagination	Implement cursor-based pagination (better for large datasets than offset).

2.10 CSV Export Stream data directly to response—never load entire tables into memory.
2.11 Rate Limiting Add `express-rate-limit` to prevent API abuse.
2.12 Security Headers Use `helmet` middleware.
2.13 Centralized Error Handling One middleware catches all errors and returns structured responses.
2.14 Structured Logging Replace all `console.log` with `pino` or `winston`.

Phase 3: Frontend Development (Days 11-17)

Step Task Key Requirements
----- ----- -----
3.1 Next.js App Router Setup Use proper folder structure: `app/(dashboard)/`, `app/api/`, `components/`, `lib/`.
3.2 Authentication Flow Login → receive tokens → store in HTTP-only cookies → attach to every request.
3.3 React Query Integration Use TanStack Query for all server state (caching, background updates, synchronization).
3.4 Form Handling Use React Hook Form + Zod for strict validation and error handling.
3.5 Dashboard UI Build charts and summary cards using aggregated API data.
3.6 Transaction Management Forms for create/edit/delete with optimistic updates.
3.7 CSV Export Button Trigger backend stream and handle file download.
3.8 Error Boundaries Wrap components to prevent entire app crashes from single component failures.
3.9 Memoization Use `React.memo`, `useMemo`, `useCallback` to prevent unnecessary re-renders.

Phase 4: DevOps & Deployment (Days 18-20)

Step Task Platform/Tool
----- ----- -----

| 4.1 | GitHub Actions CI/CD | Create `.github/workflows/ci.yml` with steps: install → lint → build → test → Prisma migrate check. |

| 4.2 | Deploy Frontend | Push to Vercel (auto-deploys from GitHub). |

| 4.3 | Deploy Backend | Push to Render. Configure environment variables. |

| 4.4 | Database Setup | Use Supabase or AWS RDS. Enable connection pooling (PgBouncer). |

| 4.5 | Environment Variables | Store secrets in platform dashboards, never in code. |

Phase 5: Testing & Optimization (Days 21-23)

Step	Task	Focus
5.1	Load Testing	Simulate 10K concurrent users with tools like k6 or Artillery.
5.2	Query Optimization	Check for N+1 queries using Prisma logs. Add missing indexes.
5.3	Security Audit	Test JWT expiration, token rotation, rate limiting, SQL injection attempts.
5.4	Final Documentation	Write architecture document explaining all technical decisions.

🧠 Necessary Knowledge & Skills

Critical Must-Know Concepts

Topic	Why It Matters	Resources to Learn
JWT & Refresh Tokens	Authentication is the foundation. Token rotation prevents replay attacks.	JWT.io documentation, Auth0 guides
Multi-Tenancy Patterns	Data isolation is non-negotiable. One mistake leaks all customer data.	Row-level security, tenant ID filtering

| Prisma ORM | Type-safe database access, migrations, and transactions. | Prisma official docs |

| Database Indexing | Without indexes on `orgId` and `createdAt`, the app will crash at scale. | PostgreSQL indexing docs |

| React Query (TanStack) | Handles caching, synchronization, and background updates automatically. | TanStack Query docs |

| Zod Validation | Type-safe schema validation that catches bad data before it hits the DB. | Zod documentation |

| Express Middleware Pattern | Centralized auth, error handling, rate limiting all use middleware. | Express.js guide |

| Next.js App Router | Newer routing paradigm with server components and layouts. | Next.js 14+ docs |

| Connection Pooling | Prevents database overload under high concurrency. | PgBouncer, Supabase pooling |

| Streaming Responses | Essential for CSV export of large datasets without memory crashes. | Node.js streams |

Security Knowledge (Non-Negotiable)

- Never store JWTs in `localStorage` → XSS attacks can steal them. Use HTTP-only cookies.
- Helmet.js → Sets security headers (CSP, HSTS, X-Frame-Options).
- Rate Limiting → Prevents brute force and DDoS on API endpoints.
- SQL Injection Prevention → Prisma handles this, but raw queries must be parameterized.
- CORS Configuration → Restrict which domains can call your API.

Performance Knowledge

- N+1 Query Problem → Fetching a list, then querying each item separately. Solution: Prisma `include` or `select`.
- Cursor Pagination → `WHERE id > lastId LIMIT 50` is faster than `OFFSET` for large tables.
- Database Normalization → No duplicate data. Use foreign keys and joins.

- React Memoization → `React.memo` prevents re-renders when props haven't changed.

⚠ Common Pitfalls to Avoid

Pitfall	Why It's Bad	Solution
---------	--------------	----------

-----	-----	-----
-------	-------	-------

Storing tokens in `localStorage`	Vulnerable to XSS theft	HTTP-only cookies
----------------------------------	-------------------------	-------------------

Using offset pagination for large data	Gets slower as table grows	Cursor-based pagination
--	----------------------------	-------------------------

Loading entire DB into memory for CSV	Server crashes on large datasets	Stream data in chunks
---------------------------------------	----------------------------------	-----------------------

Missing `orgId` filter on queries	Cross-tenant data leak	Middleware auto-injection
-----------------------------------	------------------------	---------------------------

No database transactions for financial ops	Partial writes = corrupted data	Prisma `\$transaction`
--	---------------------------------	---------------------------

`console.log` in production	Unstructured, unsearchable logs	Use `pino` or `winston`
-----------------------------	---------------------------------	-------------------------

Skipping CI/CD pipeline	Broken code reaches production	GitHub Actions mandatory checks
-------------------------	--------------------------------	---------------------------------

📁 Recommended Folder Structure

Backend (`/backend`)

...

src/

— config/	# Database, JWT secrets
-----------	-------------------------

— controllers/	# Request/response handling
----------------	-----------------------------

— services/	# Business logic
-------------	------------------

```
|— repositories/  # Database queries (Prisma)
|— middleware/    # Auth, rate limiting, error handling
|— routes/        # API route definitions
|— utils/         # Helpers, validators
|— types/         # TypeScript interfaces
└─ prisma/
  |— schema.prisma
  └─ migrations/
...

```

Frontend (`/frontend`)

```
...
app/
|— (auth)/        # Login, register (public)
|— (dashboard)/   # Protected routes
|  |— transactions/
|  |— dashboard/
|  └─ settings/
|— api/           # Next.js API routes (if needed)
components/
|— ui/            # Reusable UI components
|— forms/         # Form components
└─ layout/        # Layout wrappers
lib/
|— api.ts         # React Query setup
|— auth.ts        # Auth helpers
└─ utils.ts       # Utilities
hooks/            # Custom React hooks
types/            # TypeScript types

```

'''

🎯 Evaluation Checklist (What Graders Will Look For)

- [] Scalability: Can it handle 10K users? (Pooling, pagination, indexes)
- [] Security: No vulnerabilities in auth, headers, or input handling
- [] Performance: No N+1 queries, optimized aggregations
- [] Code Quality: Clean separation of concerns (controller/service/repo)
- [] Error Handling: App never crashes, proper HTTP status codes
- [] Deployment: Live URLs that actually work
- [] CI/CD: Pipeline passes on every push

🕒 24-Hour War Room Schedule

HOUR 0-1: Setup & Schema (Foundation)

Goal: Database design locked, project scaffolded

Minute	Action	Commands/Details
--------	--------	------------------

-----	-----	-----
-------	-------	-------

0-15	Create monorepo structure	<code>`mkdir expense-saas && cd expense-saas`</code>
------	---------------------------	--

	<code>`npx create-next-app@latest web --typescript --tailwind --eslint --app --src-dir`</code>	
--	--	--

	<code>`mkdir api && cd api && npm init -y`</code>	
--	---	--

15-30	Install backend deps	<code>`npm install express @prisma/client bcryptjs jsonwebtoken cookie-parser helmet express-rate-limit pino zod cors`</code>
-------	----------------------	---

	<code>`npm install -D prisma @types/express @types/bcryptjs @types/jsonwebtoken @types/cookie-parser typescript ts-node nodemon`</code>	
--	---	--

30-45	Design Prisma schema Define <code>`Organization`</code> , <code>`User`</code> , <code>`Transaction`</code> , <code>`RefreshToken`</code> models	
-------	---	--

45-60	Run initial migration	<code>`npx prisma migrate dev --name init`</code>
-------	-----------------------	---

Critical Schema Requirements:

- Every model MUST have ``orgId`` field
- ``Transaction`` table MUST have indexes on ``[orgId, createdAt]``, ``[orgId, type]``
- ``User`` table MUST have ``role`` enum: ``ADMIN``, ``ACCOUNTANT``, ``USER``

HOUR 1-3: Authentication & Multi-Tenancy (Core Security)

Goal: Register, login, token rotation, org isolation working

Minute	Action
--------	--------

|-----|-----|

| 0-30 | Build auth service (register/login) |

|| - Hash passwords with bcrypt (10 rounds) |

|| - Generate access token (15 min) + refresh token (7 days) |

|| - Store refresh token hash in DB (prevent reuse) |

| 30-60 | Build auth middleware |

|| - Verify JWT from HTTP-only cookie |

|| - Attach `user` + `orgId` to request object |

| 60-75 | Build role middleware |

|| - Check `req.user.role` against required roles |

| 75-90 | Build tenant middleware |

|| - Auto-inject `orgId` into all repository queries |

| 90-120 | Test with 2 organizations |

|| - Create org A + user, org B + user |

|| - Verify user A cannot see org B's data |

Token Strategy (HTTP-only cookies ONLY):

```
````javascript
// NEVER use localStorage
res.cookie('access_token', token, {
 httpOnly: true,
 secure: true,
 sameSite: 'strict',
 maxAge: 15 * 60 * 1000
});
````
```

HOUR 3-6: Transaction CRUD + Financial Consistency

Goal: Full expense/income management with validation

| Minute | Action |
|--------|--|
| 0-30 | Build transaction repository
 - `create` with Prisma `\$transaction`
 - `update` with ownership check (`orgId` match)
 - `delete` with ownership check |
| 30-60 | Build transaction service + controller
 - Zod schema validation before DB write
 - Enforce `orgId` filtering on every query |
| 60-90 | Build API routes
 - `POST /api/transactions`
 - `GET /api/transactions` (with pagination)
 - `PUT /api/transactions/:id`
 - `DELETE /api/transactions/:id` |
| 90-120 | Test with Postman/Insomnia
 - Verify CRUD works
 - Verify cross-tenant access is blocked (403) |

Zod Validation Example:

```
````javascript
const transactionSchema = z.object({
 amount: z.number().positive(),
 type: z.enum(['INCOME', 'EXPENSE']),
 description: z.string().min(1).max(500),
 category: z.string().min(1),
 date: z.string().datetime()
```

```
});
```

```
'''
```

```

```

### ### HOUR 6-8: Dashboard Aggregations + Filtering

Goal: Optimized summary endpoints

```
| Minute | Action |
```

```
|-----|-----|
```

```
| 0-30 | Build dashboard aggregation queries |
```

```
|| - `SUM(amount) WHERE orgId = ? GROUP BY type` |
```

```
|| - `SUM(amount) WHERE orgId = ? GROUP BY category` |
```

```
|| - Monthly trends with `DATE_TRUNC('month', createdAt)` |
```

```
| 30-60 | Add filtering support |
```

```
|| - `?type=EXPENSE&category=Food&startDate=...&endDate=...` |
```

```
|| - All filters MUST include `orgId` |
```

```
| 60-90 | Add pagination |
```

```
|| - Cursor-based: `?cursor=lastId&limit=20` |
```

```
|| - Fallback to offset if cursor not provided |
```

```
| 90-120 | Verify indexes are used |
```

```
|| - Run `EXPLAIN ANALYZE` on heavy queries |
```

```

```

### ### HOUR 8-10: CSV Export (Streaming)

Goal: Memory-safe large dataset export

```
| Minute | Action |
```

|                                                      |                                        |
|------------------------------------------------------|----------------------------------------|
| -----                                                | -----                                  |
| 0-30                                                 | Implement streaming CSV endpoint       |
| - Use `fast-csv` or manual string building           |                                        |
| - Stream directly to response, NO array accumulation |                                        |
| 30-60                                                | Add filtering to CSV export            |
| - Same filters as list endpoint                      |                                        |
| - Respect `orgId` isolation                          |                                        |
| 60-90                                                | Test with 10K+ records                 |
| - Verify memory stays flat during export             |                                        |
| 90-120                                               | Add rate limiting specifically for CSV |
| - `express-rate-limit`: 5 requests per hour per user |                                        |

Streaming Pattern:

```

````javascript
res.setHeader('Content-Type', 'text/csv');
res.setHeader('Content-Disposition', 'attachment; filename="transactions.csv"');

// Stream from DB cursor, never load all into memory
const stream = prisma.$queryRawUnsafe(`SELECT * FROM transactions WHERE orgId =
$1`, orgId);

// Pipe to csv formatter, then to response
```

```

### HOUR 10-12: Backend Hardening

Goal: Security, error handling, logging

|        |        |
|--------|--------|
| Minute | Action |
|--------|--------|

|-----|-----|

| 0-20 | Add `helmet` middleware |

|| - Content Security Policy, HSTS, X-Frame-Options |

| 20-40 | Add global rate limiting |

|| - General API: 100 req/15min per IP |

|| - Auth endpoints: 10 req/15min per IP |

| 40-60 | Add centralized error handler |

|| - Catch all errors, log with pino, return JSON |

|| - Never leak stack traces to client |

| 60-80 | Replace all `console.log` with `pino` |

|| - Structured JSON logs |

| 80-100 | Add CORS configuration |

|| - Whitelist only frontend domain |

| 100-120 | Final backend testing |

|| - Run all endpoints, verify responses |

---

### ### HOUR 12-14: Frontend Scaffold + Auth UI

Goal: Next.js app with login/register pages

| Minute | Action |

|-----|-----|

| 0-20 | Set up React Query provider |

|| - `QueryClient` with staleTime configuration |

| 20-40 | Build login page |

|| - React Hook Form + Zod validation |

|| - HTTP-only cookie handling (browser auto-sends) |

| 40-60 | Build register page |

| | - Org creation + user registration flow |  
| 60-80 | Build auth context/hook |  
| | - Check auth status, handle token refresh |  
| 80-100 | Add route guards |  
| | - Redirect unauthenticated to login |  
| | - Redirect authenticated to dashboard |  
| 100-120 | Test auth flow end-to-end |

---

### ### HOUR 14-17: Frontend Dashboard + Transactions

Goal: Working UI for core features

| Minute | Action |  
|-----|-----|  
| 0-30 | Build dashboard page |  
| | - Fetch aggregates from `/api/dashboard` |  
| | - Display summary cards (total income/expense/balance) |  
| | - Simple bar chart for monthly trends |  
| 30-60 | Build transaction list |  
| | - React Query with caching |  
| | - Cursor pagination (Load More button) |  
| | - Filtering UI (type, category, date range) |  
| 60-90 | Build transaction form (create/edit) |  
| | - Modal or page with validation |  
| | - Optimistic updates with React Query |  
| 90-120 | Add CSV export button |  
| | - Triggers download from streaming endpoint |

---

### ### HOUR 17-19: Frontend Polish + Error Boundaries

Goal: No crashes, good UX

| Minute | Action                                       |
|--------|----------------------------------------------|
| -----  | -----                                        |
| 0-30   | Add Error Boundary                           |
|        | - Wrap main layout, show fallback UI         |
|        | - Log errors to backend                      |
| 30-60  | Add loading states                           |
|        | - Skeleton screens for dashboard             |
|        | - Loading spinners for buttons               |
| 60-90  | Add toast notifications                      |
|        | - Success/error feedback for CRUD operations |
| 90-120 | Memoization pass                             |
|        | - `React.memo` on list items                 |
|        | - `useMemo` for computed values              |
|        | - `useCallback` for event handlers           |

---

### ### HOUR 19-21: CI/CD + GitHub Setup

Goal: Pipeline running on every push

| Minute | Action                            |
|--------|-----------------------------------|
| -----  | -----                             |
| 0-30   | Create `.github/workflows/ci.yml` |
|        | - Step 1: Checkout code           |

| | - Step 2: Install dependencies (both web and api) |  
| | - Step 3: Run ESLint |  
| | - Step 4: Build both projects |  
| | - Step 5: Run tests (if any) |  
| | - Step 6: Prisma migrate check (`prisma migrate status`) |  
| 30-60 | Commit everything with clean history |  
| | - `git init`, meaningful commits by feature |  
| 60-90 | Push to GitHub |  
| | - Verify Actions tab shows pipeline running |  
| 90-120 | Fix any pipeline failures |  
| | - Common issues: Node version, env vars, build errors |

Sample CI/CD YAML:

```
``yaml
name: CI/CD
on: [push]
jobs:
 build:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-node@v4
 with:
 node-version: '20'
 - run: cd api && npm ci && npx prisma generate && npm run lint && npm run build
 - run: cd web && npm ci && npm run lint && npm run build
 - run: cd api && npx prisma migrate status
``
```

---

### ### HOUR 21-23: Deployment

Goal: Live URLs

| Minute                                                         | Action                                           |
|----------------------------------------------------------------|--------------------------------------------------|
| 0-30                                                           | Deploy frontend to Vercel                        |
| - Connect GitHub repo                                          |                                                  |
| - Set environment variables (API_URL)                          |                                                  |
| - Deploy                                                       |                                                  |
| 30-60                                                          | Deploy backend to Render                         |
| - Connect GitHub repo                                          |                                                  |
| - Set environment variables (DATABASE_URL, JWT_SECRET, etc.)   |                                                  |
| - Deploy                                                       |                                                  |
| 60-90                                                          | Set up managed PostgreSQL                        |
| - Supabase (fastest) or RDS                                    |                                                  |
| - Run `prisma migrate deploy` against production DB            |                                                  |
| - Enable connection pooling (Supabase does this automatically) |                                                  |
| 90-120                                                         | Update frontend env vars with production API URL |
| - Redeploy if needed                                           |                                                  |
| - Test live URLs                                               |                                                  |

Required Environment Variables:

```
```bash
```

```
# Backend (.env)
```

```
DATABASE_URL="postgresql://..."
```

```
JWT_SECRET="random-256-bit-string"
```

```
JWT_REFRESH_SECRET="different-random-string"
```

```
FRONTEND_URL="https://your-app.vercel.app"
```

```
# Frontend (.env.local)
```

```
NEXT_PUBLIC_API_URL="https://your-api.onrender.com"
```

```
'''
```

```
---
```

HOUR 23-24: Final Testing + Documentation

Goal: Submission-ready

| Minute | Action |
|--------|--------|
|--------|--------|

| ----- | ----- |
|-------|-------|
|-------|-------|

| | |
|------|--------------------|
| 0-20 | End-to-end testing |
|------|--------------------|

| | |
|--|-------------------|
| | - Register 2 orgs |
|--|-------------------|

| | |
|--|-------------------------------|
| | - Create transactions in both |
|--|-------------------------------|

| | |
|--|--|
| | - Verify isolation (org A can't see org B) |
|--|--|

| | |
|--|-------------------|
| | - Test CSV export |
|--|-------------------|

| | |
|--|--|
| | - Test role restrictions (User vs Admin) |
|--|--|

| | |
|-------|-------------------|
| 20-40 | Performance check |
|-------|-------------------|

| | |
|--|--|
| | - Run 100 concurrent requests to dashboard |
|--|--|

| | |
|--|---------------------------------|
| | - Verify response times < 200ms |
|--|---------------------------------|

| | |
|-------|----------------|
| 40-60 | Security check |
|-------|----------------|

| | |
|--|--------------------------------------|
| | - Verify no localStorage token usage |
|--|--------------------------------------|

| | |
|--|---------------------------------|
| | - Verify helmet headers present |
|--|---------------------------------|

| | |
|--|-------------------------------------|
| | - Verify rate limiting blocks abuse |
|--|-------------------------------------|

| | |
|-------|-----------------------------|
| 60-80 | Write Architecture Document |
|-------|-----------------------------|

| | |
|--|--|
| | - 1-page explaining: tech stack, multi-tenancy approach, auth strategy, deployment diagram |
|--|--|

| | |
|--|--|
| | |
|--|--|

| |
|--|
| 80-100 Clean up |
| - Remove all `console.log` |
| - Verify structured logging works |
| 100-120 Final commit + push |
| - Tag release: `git tag -a v1.0 -m "Submission"` |
| - Submit GitHub repo + live URLs |

🚨 24-Hour Survival Rules

| Rule Why |
|---|
| ----- ----- |
| NO custom UI library Use shadcn/ui or plain Tailwind. Save 4+ hours. |
| NO complex charts Use simple div bars or recharts basic. Skip D3. |
| NO email service Skip email verification. Use immediate activation. |
| NO payment integration Skip Stripe. This is an internal tool demo. |
| NO real-time features Skip WebSockets. Polling or manual refresh is fine. |
| NO unit tests Skip Jest. Manual testing only in 24h. (Add TODO comments) |
| Use Supabase over RDS Faster setup, free tier, built-in pooling. |
| Use Render over AWS Zero-config deployment from GitHub. |

📦 Tech Stack (Locked - No Debates)

| Layer Tool Reason |
|--|
| ----- ----- ----- |
| Frontend Next.js 14 (App Router) Full-stack capable, fast deployment |

| Styling | Tailwind CSS | Zero config, fast iteration |
| Components | shadcn/ui | Copy-paste, accessible, beautiful |
| Forms | React Hook Form + Zod | Best DX, type-safe validation |
| State | TanStack Query (React Query) | Caching, synchronization built-in |
| Backend | Express.js | Fastest to scaffold, well-documented |
| ORM | Prisma | Type-safe, migrations, transactions |
| Database | PostgreSQL (Supabase) | Free, managed, pooled connections |
| Auth | JWT + bcrypt | Stateless, fast, well-understood |
| Logging | pino | Structured, fast, JSON output |
| Deployment | Vercel + Render | Git-push deploy, free tiers |

Hour-by-Hour Deliverables Checklist

- [] Hour 1: Prisma schema committed, DB migrated
- [] Hour 3: Register/login API tested, tokens working
- [] Hour 6: Transaction CRUD tested with tenant isolation
- [] Hour 8: Dashboard API returns aggregates < 100ms
- [] Hour 10: CSV export streams 10K records without memory spike
- [] Hour 12: Backend has helmet, rate limiting, pino logging
- [] Hour 14: Frontend auth pages work, cookies set correctly
- [] Hour 17: Dashboard shows data, transactions CRUD works in UI
- [] Hour 19: Error boundaries catch errors, no white screens
- [] Hour 21: GitHub Actions pipeline passes on push
- [] Hour 23: Live URLs work, multi-tenancy verified
- [] Hour 24: Architecture doc written, repo submitted

⚡ If Running Behind Schedule (Cut List)

If behind by >30 minutes, cut these in order:

1. CSV Export → Replace with "Export JSON" (simpler, still proves streaming concept)
2. Dashboard Charts → Show numbers only (cards with totals)
3. Role-Based UI → Hide admin features, enforce at API only
4. Error Boundaries → Add basic `try/catch` in components
5. React Query Caching → Use basic `fetch` with `useEffect` (sacrilege, but saves time)
6. Refresh Token Rotation → Use long-lived access tokens (security risk, but functional)

NEVER CUT:

- Multi-tenancy isolation
- Password hashing
- `orgId` filtering on every query
- Database transactions for writes
- Helmet security headers

🎯 Final Submission Checklist

- [] GitHub repo link with clean commit history
- [] Live frontend URL (Vercel)
- [] Live API URL (Render)
- [] Prisma schema file in repo
- [] Architecture document (PDF or Markdown)
- [] README with setup instructions

- [] All environment variables documented (without secrets)